Advanced Databases

2026/2027

Prof. Márcia Barros

Project Description:
Comparative Study of MySQL and MongoDB

Overview

This project focuses on the design, implementation, and optimisation of a relational database in **MySQL** and a document database in **MongoDB**, using the **same dataset**.

Students will design database schemas, implement queries and transactions, test concurrent access, and evaluate the impact of indexing and query optimisation. The objective is to understand how different database models affect data organisation, access, and performance.

Project Description

The students will implement two databases that represent the same information: one using MySQL and another using MongoDB.

The databases may have different structures, reflecting the characteristics of each model. However, they must support equivalent queries and operations so that their behaviour and performance can be compared.

You are encouraged to choose a domain of personal or academic interest, such as courses, books, movies, products, transport, or scientific data.

The project is divided into four parts, demonstrated during the practical classes.

Infrastructure

- **Relational database:** MySQL.
- **NoSQL document database:** MongoDB.
- **Programming language:** Python, using the libraries introduced in class, such as `pymongo`, `mysql.connector`, `sqlalchemy`, and `pandas`.

Data Source

- Select a publicly available dataset, for example from Kaggle (`https://www.kaggle.com/`).
- The dataset must:
  - Be suitable for creating a relational database with at least three related tables/collections.
  - Support a document-oriented representation with justified embedding or referencing decisions.
  - Contain 100,000 records in its main data component (the original dataset may be bigger but you should sample it to 100k records).

  - Support varied queries, aggregations, and data modification operations.
- Both databases must contain the same underlying information.
- Identify the dataset source and document any cleaning or transformations.
- Multiple sources of data is a plus.

Example dataset domains:

- Online retail transactions.
- Movie ratings and metadata.
- Book ratings and metadata.
- Transport or mobility records.

Project Goals and Tasks

Goal 1 — Selection of the Dataset and Database Design

- Select and analyze the dataset, identifying its entities, relationships, and data hierarchies.
- Define an application context and the main information needs that the databases will support.
- Design a relational schema for MySQL, identifying:
  - Tables and attributes.
  - Primary and foreign keys.
  - Relationships and integrity constraints.
- Design a document schema for MongoDB, identifying:
  - Collections and document structures.
  - Embedded documents and references.
  - Relevant validation rules.
- Justify the modelling decisions according to the expected queries and operations.
- Explain how the same information is represented in both databases.

Deliverables (to show in class):

- Dataset description and source.
- ER diagram and relational schema.
- MongoDB document model, including example documents.
- Justification of the design decisions.

Goal 2 — Database Implementation and Data Ingestion

- Implement the MySQL and MongoDB databases according to the proposed designs.
- Develop scripts to clean and transform the data where necessary.
- Import the same dataset into both databases.
- Verify that both implementations contain equivalent information, using counts and selected consistency checks.
- Measure data ingestion time for each database and document the loading method used.

- Make the database creation and ingestion processes reproducible.

Deliverables (to show in class):

- MySQL database creation scripts.
- MongoDB collection creation and validation scripts.
- Data preparation and loading scripts.
- Evidence of data consistency between the two implementations.
- Comparison of data ingestion times.

Goal 3 — Queries, Updates and Transactions

- Design and implement equivalent queries in MySQL and MongoDB, including:
  - Simple filtering and projection.
  - Sorting.
  - Aggregations and grouping.
  - Queries involving related entities, involving several tables/collections in the same query.
- Implement insert, update, and delete operations in both databases.
- Implement a transaction involving multiple related changes in each system.
- Test successful execution and rollback when an operation fails.
- Develop scripts that simulate multiple clients accessing and modifying data concurrently.
- Observe response times, conflicts, and the consistency of the resulting data.
- Verify that equivalent operations produce equivalent results in both databases.

Deliverables (to show in class):

- Documented SQL queries and MongoDB queries or aggregation pipelines.
- Examples of outputs and correctness checks.
- Insert, update, and delete operations.
- Transaction examples, including rollback.
- Concurrency test scripts and a summary of the observed behaviour.

Goal 4 — Indexing, Query Plans and Performance Evaluation

- Establish baseline performance measurements for the workload developed in Goal 3.
- Document the indexes already present in each database, including automatically created indexes.
- Propose and implement additional indexes based on the queries and access patterns.
- Inspect and explain relevant query plans before and after optimisation.
- Apply appropriate query or schema improvements, justifying each change.
- Repeat the performance measurements after optimisation.
- Compare:
  - Query execution times.
  - The impact of indexes on insert, update, and delete operations.

  - Performance under concurrent access.
  - Storage requirements before and after adding indexes.
- Discuss which database performs better for each type of operation and explain the observed trade-offs.

Deliverables (to show in class):

- Index definitions and justification.
- Relevant query plans and their interpretation.
- Benchmarking scripts and description of the experimental setup.
- Tables or plots comparing performance before and after optimisation.
- Discussion of the results and limitations.

Final Delivery

Date: December 6th, 2026 - 23:59h

Submission platform: Moodle.

Submit a ZIP file containing the code and report.

ZIP filename: `BDA2627_GroupNumber.zip`, for example, `BDA2627_G01.zip`.

The report

- Maximum of **8 pages and 1,500 words**.
- Organised by the four project goals.
- Identify all group members.
- Describe and justify the database designs.
- Discuss the tasks completed and any requirements not fulfilled.
- Present the experimental setup, performance results, and conclusions.
- Explain how to reproduce the project, including database creation, data ingestion, query execution, and performance tests.

AI Usage Policy for Student Projects

Students may use AI tools and platforms, such as ChatGPT, Copilot, or Gemini, to assist with their projects. However, all students are fully responsible for understanding and being able to explain every part of the code, text, and information submitted.

AI tools may not be used to generate an entire project or major sections of it. If a project is fully AI-generated, or if a student cannot explain their own code or content, the project will receive a grade of zero.

Students must be able to demonstrate how their implementation works and explain their design decisions, queries, transactions, and experimental results.